Table des matières

Exercices corrigés 03 — Monter le projet dbt et le STAGING 1
Questions courtes 1
Questions pratiques 1
Diagnostics 2
Questions à développer 2
Mises en situation 2
Corrigé 3

Exercices corrigés 03 — Monter le projet dbt et le STAGING

Comment travailler ces exercices. Répondez seul, puis vérifiez avec le corrigé. Les questions pratiques se tapent dans la première fenêtre du labo, placée dans `Projet_Boreal_intermediaire` avec `DBT_PROJECT_DIR=02_ae`, sur le projet tel que le Labo 03 l'a laissé. Tout vient de la Théorie 03 et du Labo 03.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Les gestes refusés. Quels sont les deux gestes que le `STAGING` refuse, et la raison de fond de chacun ?
2. Nommer en dur. Un modèle qui écrit `from staging.stg_commandes` au lieu de `ref()` fonctionne. Qu'est-ce qui casse, alors ?

Questions pratiques

P1. Voir le SQL compilé. Donnez les deux commandes qui produisent puis affichent le SQL réellement envoyé à DuckDB pour `stg_articles`, et dites ce que `source()` y est devenu.

P2. Lire l'aval. Donnez la commande qui liste `stg_commandes` et tout ce qui en dépend, et son résultat sur le projet du labo.

© 2026 iAlexMG. Tous droits réservés.

Diagnostics

E1. Le staging naïf. Un collègue écrit sa vue de staging des commandes ainsi :

```
select
    vbeln as numero_commande,
    kunnr as code_concessionnaire,
    netwr::decimal(18, 2) as valeur_nette
from {{ source('raw', 'erp_vbak') }}
```

Elle tourne, et le total du chiffre d'affaires part en production. Dites ce qui manque, de combien le total est faux, et pourquoi personne ne le verra.

E2. Le dossier de trop. Une collègue a suivi le labo. Le lendemain, elle ouvre l'Anaconda Prompt, se place dans `Projet_Boreal_intermediaire\02_ae` « pour être dans le projet », et lance `dbt debug`, qui répond `profiles.yml file [ERROR not found]`. Dites la cause et le geste qui corrige.

Questions à développer

A. Le staging qui ne joint pas. La règle paraît arbitraire, et elle coûte du travail : sans jointure, chaque consommateur du staging doit refaire les siennes. Défendez la règle, puis donnez le cas limite où vous accepteriez de la contourner, et à quelles conditions.

Mises en situation

M1. Fromagerie de Saint-Prime. La fromagerie reçoit chaque semaine, de son distributeur, un fichier des ventes en épicerie : point de vente, produit, semaine, quantité. Le distributeur renvoie le fichier complet des douze dernières semaines à chaque livraison, si bien que la même semaine arrive douze fois, parfois avec des chiffres révisés. On vous demande de monter le staging. Dites ce que vous écrivez, quelle colonne fait office de lot, et le contrôle que vous posez avant de livrer quoi que ce soit à l'aval.

© 2026 iAlexMG. Tous droits réservés.

Corrigé

Comment lire ce corrigé. Il donne la réponse et le pourquoi, jamais une notion neuve : tout ce qu'il emploie a été enseigné dans la Théorie 03 ou pratiqué dans le Labo 03. Les chiffres sont ceux de la `RAW` montée par le Cours 00.

Questions courtes

1. Les gestes refusés.

- Retirer des lignes parce qu'elles ne compteraient pas pour le métier, comme les concessionnaires à la fiche fermée ou les postes refusés : décider ce qui compte revient aux couches suivantes, et une ligne retirée au staging manque à tous ceux qui en dépendent. Tirer l'état courant ne retire que des copies.

- Joindre une autre table. Une jointure est une décision de modélisation, et la modélisation a ses couches : un `STAGING` qui joint devient un endroit où le métier se fait, en dehors de l'endroit prévu pour lui. *Théorie : `Theorie_03_AE.md`, section 6.*

2. Nommer en dur. Le graphe. dbt ne voit pas le lien : il peut bâtir les deux modèles dans le mauvais ordre, ne sait plus quoi rebâtir quand `stg_commandes` change, et la documentation ment sur le lignage. Le nom du schéma, écrit en dur, empêche aussi de changer d'environnement. *Théorie : `Theorie_03_AE.md`, section 9.*

Questions pratiques

P1. Voir le SQL compilé.

```
dbt compile --select stg_articles
type 02_ae\target\compiled\boreal_ae\models\staging\stg_articles.sql
```

Le fichier compilé montre `{{ source('raw', 'erp_mara') }}` devenu `"boreal"."raw"."erp_mara"` : la base, le schéma et la table, la seule forme que l'entrepôt comprend. C'est le premier endroit où regarder quand un modèle ne rend pas ce qu'on attend. *Théorie : `Theorie_03_AE.md`, sections 1 et 4.*

P2. Lire l'aval.

```
dbt ls --select stg_commandes+
```

Attendu, deux lignes : `boreal_ae.staging.ctl_commandes_sans_concessionnaire` et `boreal_ae.staging.stg_commandes`. Le `+` après un nom veut dire « et tout ce qui en dépend » : la commande répond à « qu'est-ce que je casse si je change ce modèle ». *Théorie : `Theorie_03_AE.md`, section 9.*

© 2026 iAlexMG. Tous droits réservés.

Diagnostics

E1. Le staging naïf. Il manque l'état courant : la vue lit les 26 538 lignes de la `RAW` au lieu des 25 483 commandes. La somme des valeurs nettes passe de 430 148 012,16 $ à 453 849 040,59 $, soit 23 701 028,43 $ de trop, 5,51 %. Personne ne le verra : 453 millions est un chiffre d'affaires plausible pour un fabricant de véhicules récréatifs, et rien ne le contredit, jusqu'au jour où quelqu'un recompte depuis l'ERP. *Théorie : `Theorie_03_AE.md`, section 6.*

E2. Le dossier de trop. dbt cherche `profiles.yml` dans le dossier d'où on le lance, puis dans le dossier `.dbt` du répertoire personnel. Depuis `02_ae`, il ne trouve ni l'un ni l'autre : la connexion vit dans `Projet_Boreal_intermediaire`, hors du dépôt, et son chemin relatif se lit de là. Le geste : revenir dans `Projet_Boreal_intermediaire`, redonner `set DBT_PROJECT_DIR=02_ae`, et relancer. La fenêtre ne quitte plus ce dossier. *Théorie : `Theorie_03_AE.md`, section 3.*

Questions à développer

A. Le staging qui ne joint pas. La défense tient en trois arguments. Une vue de staging qui joint n'a plus de grain clair : elle décrit une commande, ou un concessionnaire, ou les deux, et personne ne sait plus ce qu'une ligne représente. Elle fige des choix qui appartiennent à la modélisation, comme la version d'une dimension : le jour où le concessionnaire passe en type 2, au Cours 05, tout ce qui consomme cette vue hérite du nom d'aujourd'hui. Et elle rend le staging moins réutilisable : un consommateur qui n'a pas besoin du concessionnaire paie quand même la jointure. Le cas limite acceptable est une source livrée en deux fichiers qui ne forment qu'une entité logique, un en-tête et son détail coupés par l'exportateur, qui n'ont aucun sens séparément : les réunir reconstitue une source. La condition est de pouvoir dire « ces deux fichiers sont une seule table que l'exportateur a coupée » ; sinon, c'est une jointure de modélisation, et elle appartient à l'`INTERMEDIATE`. *Théorie : `Theorie_03_AE.md`, section 6.*

Mises en situation

M1. Fromagerie de Saint-Prime. Le cas est celui de Boréal, en plus systématique : chaque semaine arrive douze fois, avec des chiffres qui peuvent différer d'une livraison à l'autre. J'écris une vue de staging qui tire l'état courant, avec pour clé le triplet point de vente, produit et semaine, et pour ordre la livraison la plus récente. La colonne qui fait office de lot est la date de réception du fichier ; si le distributeur ne la fournit pas, c'est le nom du fichier ou l'heure du chargement qui la porte, et sans elle la règle « la plus récente gagne » ne s'applique pas : c'est la première chose à réclamer. Le contrôle avant de livrer est celui du grain, les lignes contre les triplets distincts, et une mesure des révisions à montrer au métier : si elles sont fortes, tout rapport hebdomadaire de moins de douze semaines est provisoire, et il faut le dire. *Théorie : `Theorie_03_AE.md`, sections 6 et 7.*

© 2026 iAlexMG. Tous droits réservés.